

# New Techniques in Knowledge Compilation and Their Applications

Petr Illner

Department of Theoretical Computer Science and Mathematical Logic, Faculty of Mathematics and Physics  
Charles University, Czech Republic

[illner3@gmail.com](mailto:illner3@gmail.com)

# Knowledge Representation and Knowledge Compilation

**Representation:** How to efficiently represent Boolean functions to support specific operations in polynomial time.

**Compilation:** How to convert between such representations.

**The research gap:** Lack of a practical knowledge compiler directly targeting DNNF-based representations that support polynomial-time consistency and clausal entailment, without also supporting polynomial-time model counting.

# Isomorphism-Aware Exact Model Counting

**Counting:** How to compute the number of satisfying assignments of a CNF formula.

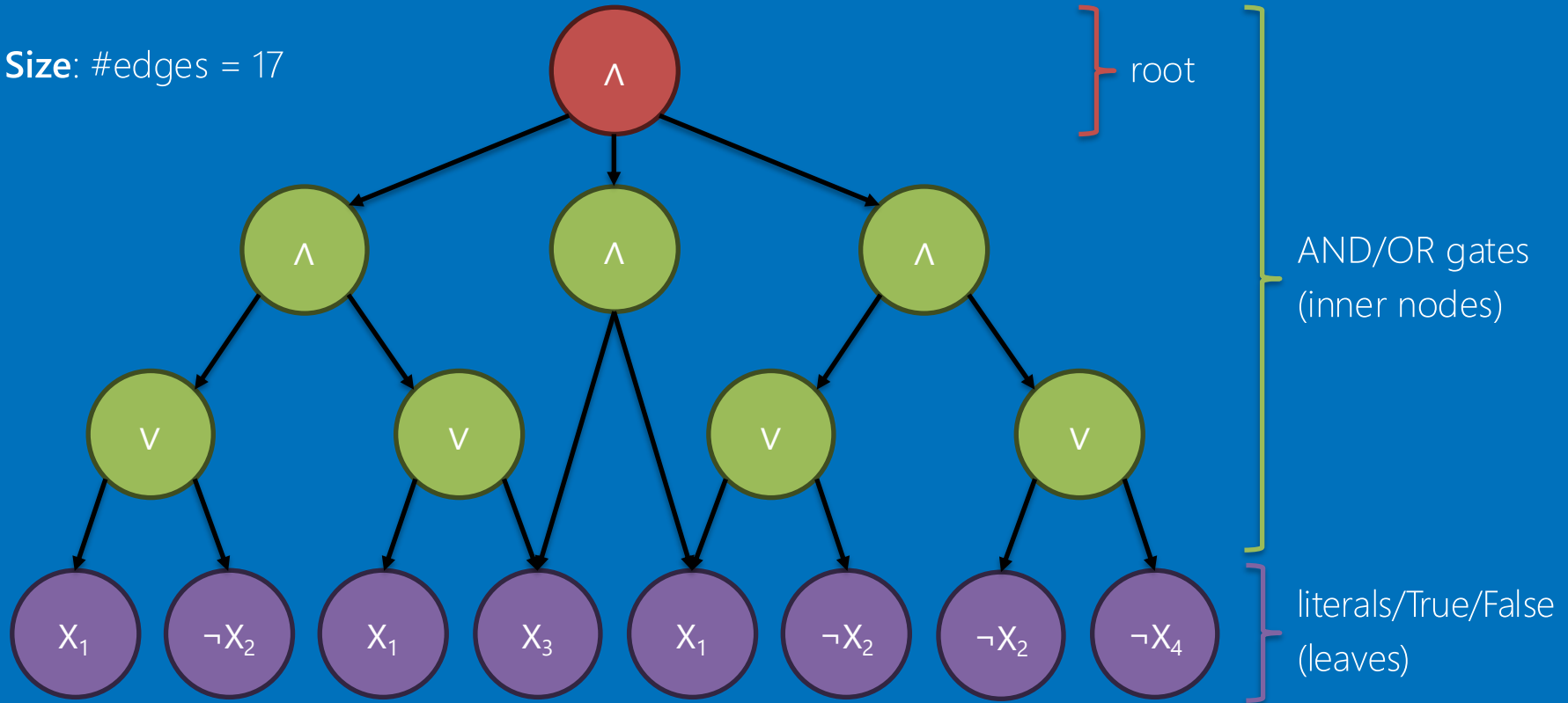
**Isomorphism-awareness:** How to reuse results for subproblems that differ only by variable renaming and consistent polarity changes.

**The research gap:** Lack of an isomorphism-aware model counter competitive with standard (non-isomorphism-aware) model counters on less symmetric instances.

# Knowledge Representation and Knowledge Compilation

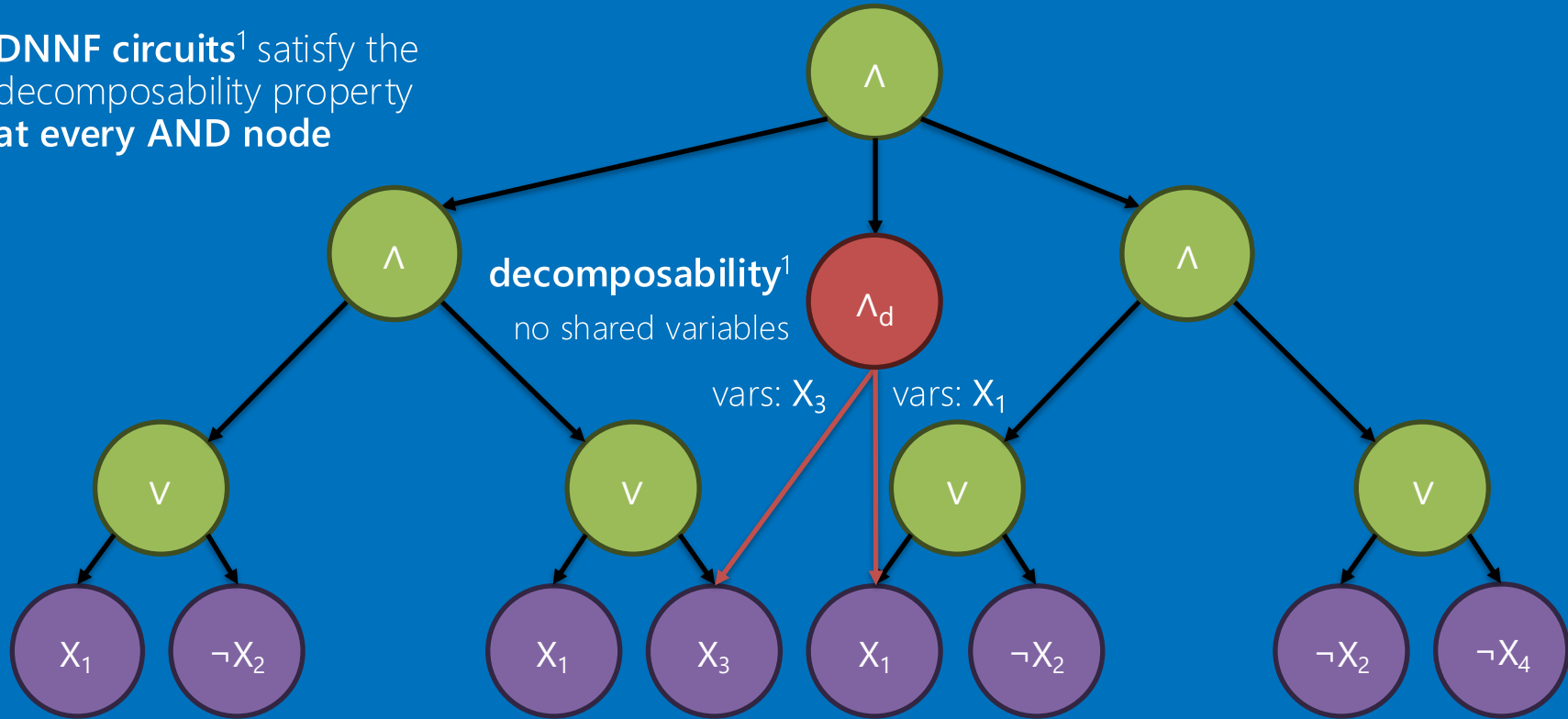
# Negation Normal Form (NNF) Circuits

**Size:** #edges = 17



# Decomposable NNF (DNNF) Circuits

**DNNF circuits**<sup>1</sup> satisfy the decomposability property **at every AND node**



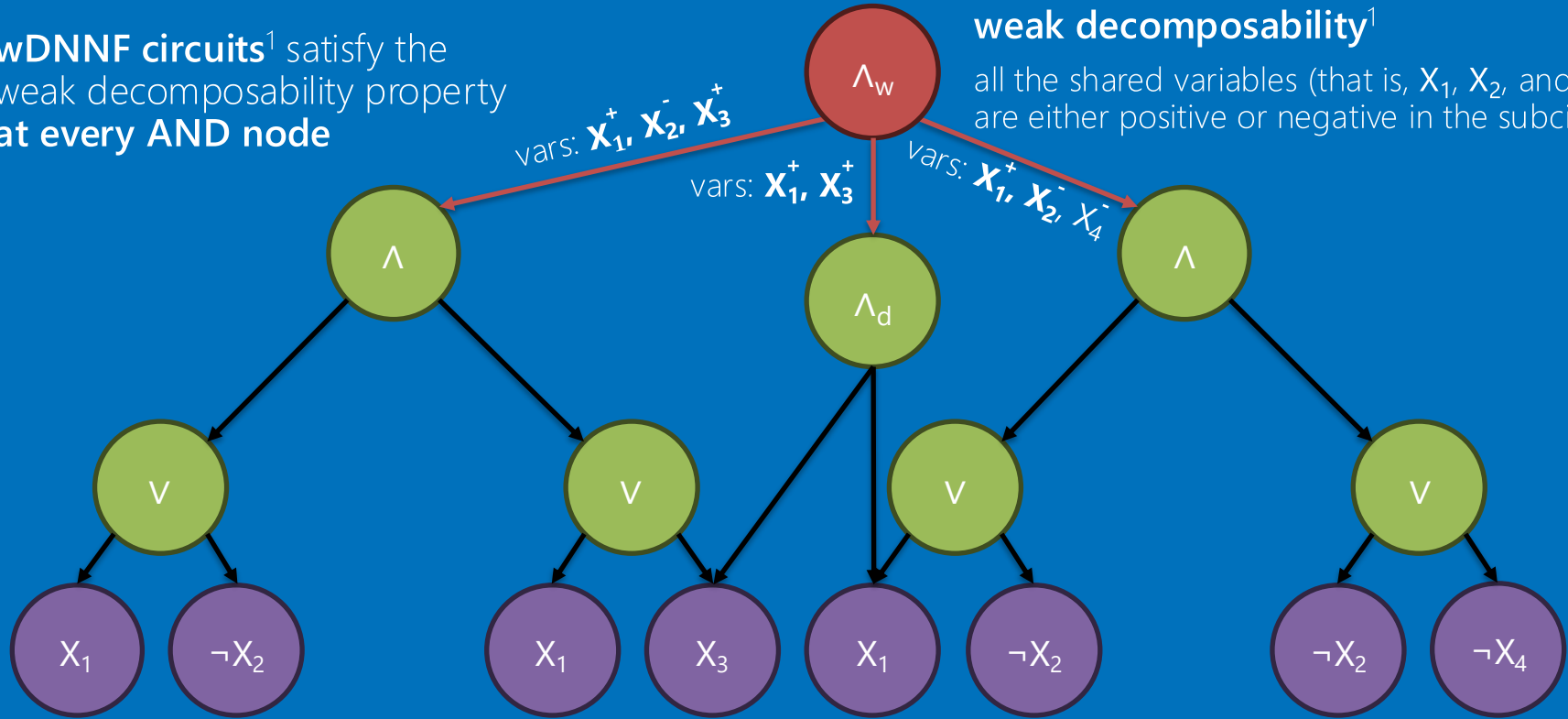
<sup>1</sup> DARWICHE, Adnan. Compiling knowledge into decomposable negation normal form. In: IJCAI. 1999, p. 284-289.

# Weak Decomposable NNF (wDNNF) Circuits

**wDNNF circuits**<sup>1</sup> satisfy the weak decomposability property **at every AND node**

**weak decomposability**<sup>1</sup>

all the shared variables (that is,  $X_1$ ,  $X_2$ , and  $X_3$ ) are either positive or negative in the subcircuits



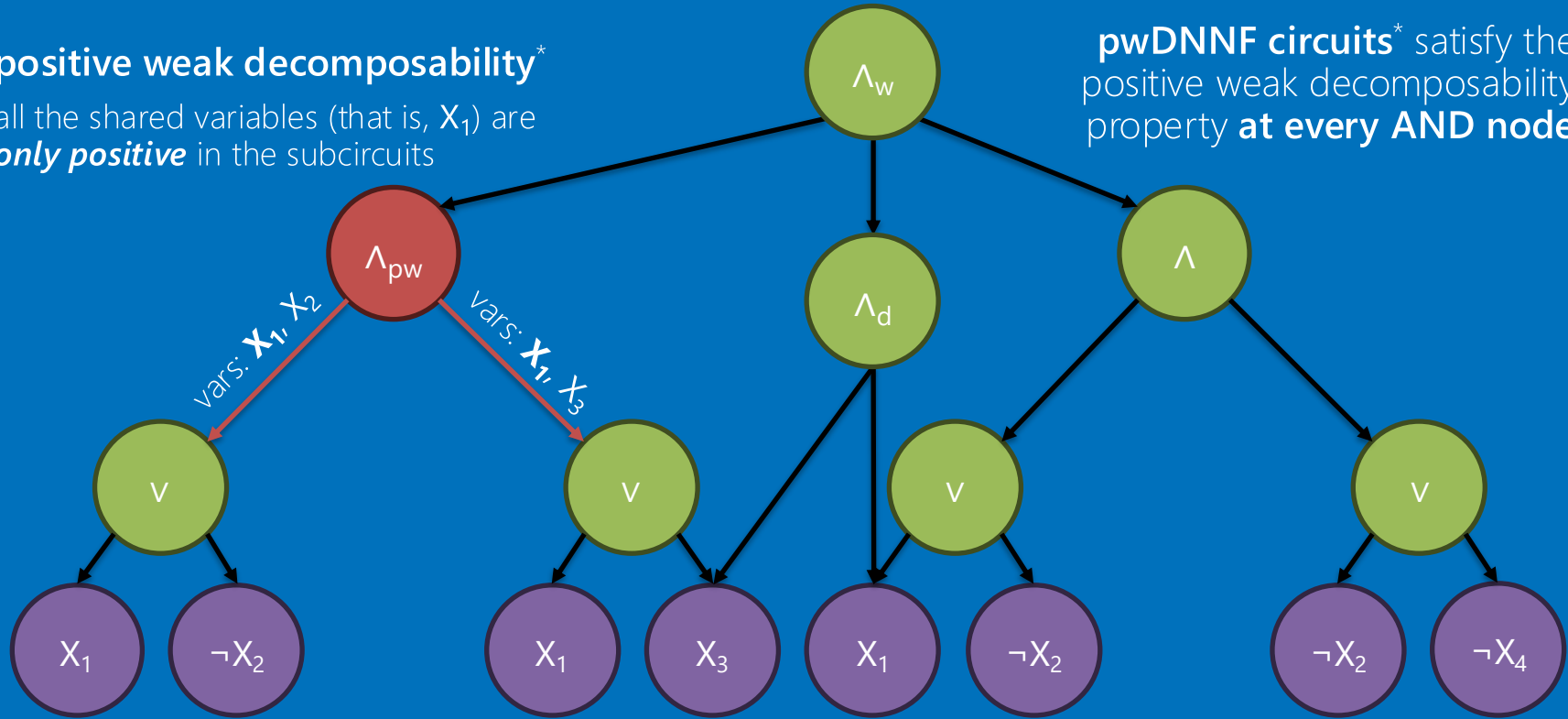
<sup>1</sup> AKSHAY, Sundararaman, et al. What's hard about Boolean functional synthesis? In: International Conference on Computer Aided Verification. Cham: Springer International Publishing, 2018.

# Positive Weak Decomposable NNF (pwDNNF) Circuits

## positive weak decomposability\*

all the shared variables (that is,  $X_1$ ) are **only positive** in the subcircuits

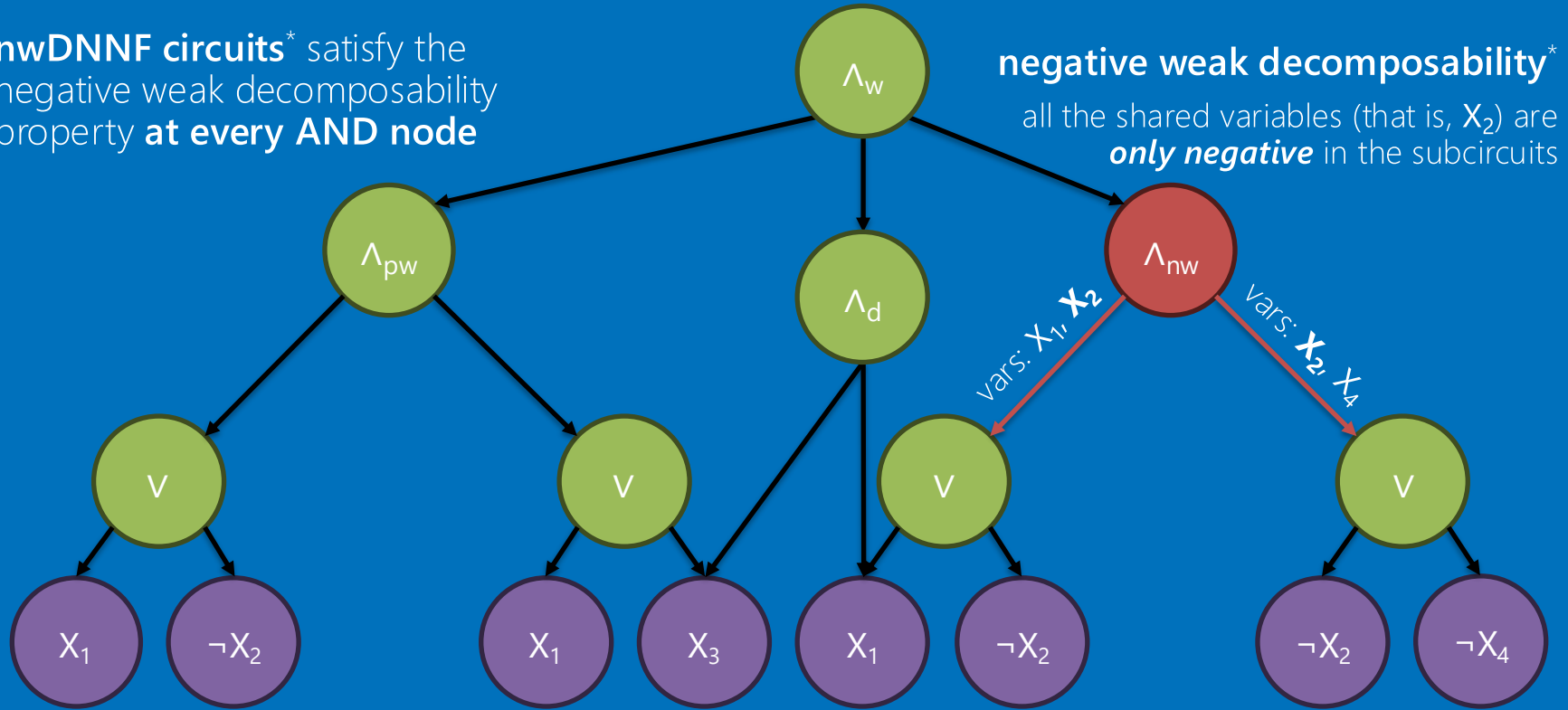
**pwDNNF circuits\*** satisfy the positive weak decomposability property **at every AND node**



\* new property / circuit type

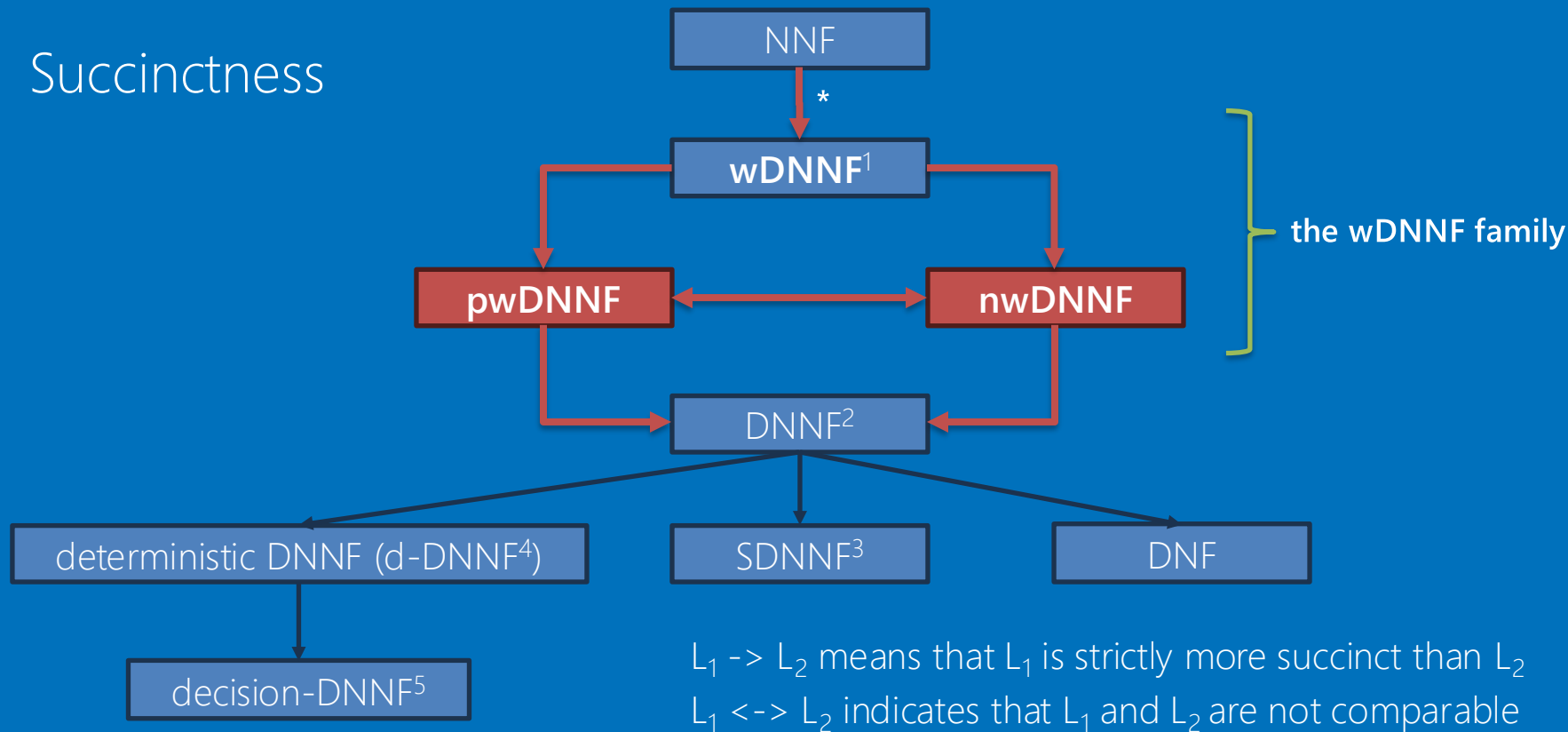
# Negative Weak Decomposable NNF (nwDNNF) Circuits

**nwDNNF circuits**<sup>\*</sup> satisfy the negative weak decomposability property **at every AND node**



<sup>\*</sup> new property / circuit type

# Succinctness



<sup>1</sup> AKSHAY, Sundararaman, et al. What's hard about Boolean functional synthesis? In: International Conference on Computer Aided Verification. Cham: Springer International Publishing, 2018.

<sup>2</sup> DARWICHE, Adnan. Compiling knowledge into decomposable negation normal form. In: IJCAI. 1999, p. 284-289.

<sup>3</sup> PIPATSRISAWAT, Knot; DARWICHE, Adnan. New Compilation Languages Based on Structured Decomposability. In: AAAI. 2008, p. 517-522.

<sup>4</sup> DARWICHE, Adnan. On the tractable counting of theory models and its application to truth maintenance and belief revision. Journal of Applied Non-Classical Logics, 2001, 11:1-2: 11-34.

<sup>5</sup> HUANG, Jinbo; DARWICHE, Adnan. The language of search. Journal of Artificial Intelligence Research, 2007, 29: 191-219.

**red = our contributions**

**\* proved by the co-author**

# Standard Queries

| Query <sup>1</sup>                      | wDNNF | pwDNNF | nwDNNF | DNNF | d-DNNF |
|-----------------------------------------|-------|--------|--------|------|--------|
| <b>Consistency</b> ( <i>CO</i> )        | ✓*    | ✓*     | ✓*     | ✓    | ✓      |
| Validity ( <i>VA</i> )                  | ✗*    | ✗*     | ✗*     | ✗    | ✓      |
| <b>Clausal entailment</b> ( <i>CE</i> ) | ✓*    | ✓*     | ✓*     | ✓    | ✓      |
| Implicant ( <i>IM</i> )                 | ✗*    | ✗*     | ✗*     | ✗    | ✓      |
| Equivalence ( <i>EQ</i> )               | ✗*    | ✗*     | ✗*     | ✗    | ?      |
| Sentential entailment ( <i>SE</i> )     | ✗*    | ✗*     | ✗*     | ✗    | ✗      |
| Model counting ( <i>CT</i> )            | ✗*    | ✗*     | ✗*     | ✗    | ✓      |
| <b>Model enumeration</b> ( <i>ME</i> )  | ✓*    | ✓*     | ✓*     | ✓    | ✓      |

✓ polytime    ✓ with polynomial delay    ✗ not polytime unless  $P = NP$     ? unknown    \* **our contributions**

<sup>1</sup> DARWICHE, Adnan; MARQUIS, Pierre. A knowledge compilation map. Journal of Artificial Intelligence Research, 2002, 17: 229-264.

# Transformations

| Transformation <sup>1</sup>              | wDNNF | pwDNNF | nwDNNF | DNNF | d-DNNF            |
|------------------------------------------|-------|--------|--------|------|-------------------|
| <b>Conditioning</b> ( $CD$ )             | ✓*    | ✓*     | ✓*     | ✓    | ✓                 |
| <b>Forgetting</b> ( $FO$ )               | ✓     | ✓*     | ✓*     | ✓    | ✗                 |
| Bounded conjunction ( $\wedge BC$ )      | ✗*    | ✗*     | ✗*     | ✗    | ✗                 |
| <b>Bounded disjunction</b> ( $\vee BC$ ) | ✓*    | ✓*     | ✓*     | ✓    | ✗                 |
| Negation ( $\neg C$ )                    | ✗*    | ✗*     | ✗*     | ✗    | ✗ <sup>2, 3</sup> |

✓ polytime    ✗ not polytime unless  $P = NP$

\* our contributions

<sup>1</sup> DARWICHE, Adnan; MARQUIS, Pierre. A knowledge compilation map. Journal of Artificial Intelligence Research, 2002, 17: 229 -264.

<sup>2</sup> Unconditional size lower bound; Van den Broeck, 2026. <https://github.com/UCLA-StarAI/ddnnf-negation>

<sup>3</sup> <https://github.com/kuldeepmeel/dnnfnegation/>

# Non-Standard Queries

| Query <sup>1,2</sup>                                | wDNNF | pwDNNF | nwDNNF | DNNF | d-DNNF |
|-----------------------------------------------------|-------|--------|--------|------|--------|
| Minimum cardinality<br>( <i>MinCard</i> )           | ✗*    | ✗*     | ✓*     | ✓    | ✓      |
| Weighted minimum<br>cardinality ( <i>wMinCard</i> ) | ✗*    | ✗*     | ✓*     | ✓    | ✓      |
| Maximum cardinality<br>( <i>MaxCard</i> )           | ✗*    | ✓*     | ✗*     | ✓    | ✓      |
| Weighted maximum<br>cardinality ( <i>wMaxCard</i> ) | ✗*    | ✓*     | ✗*     | ✓    | ✓      |

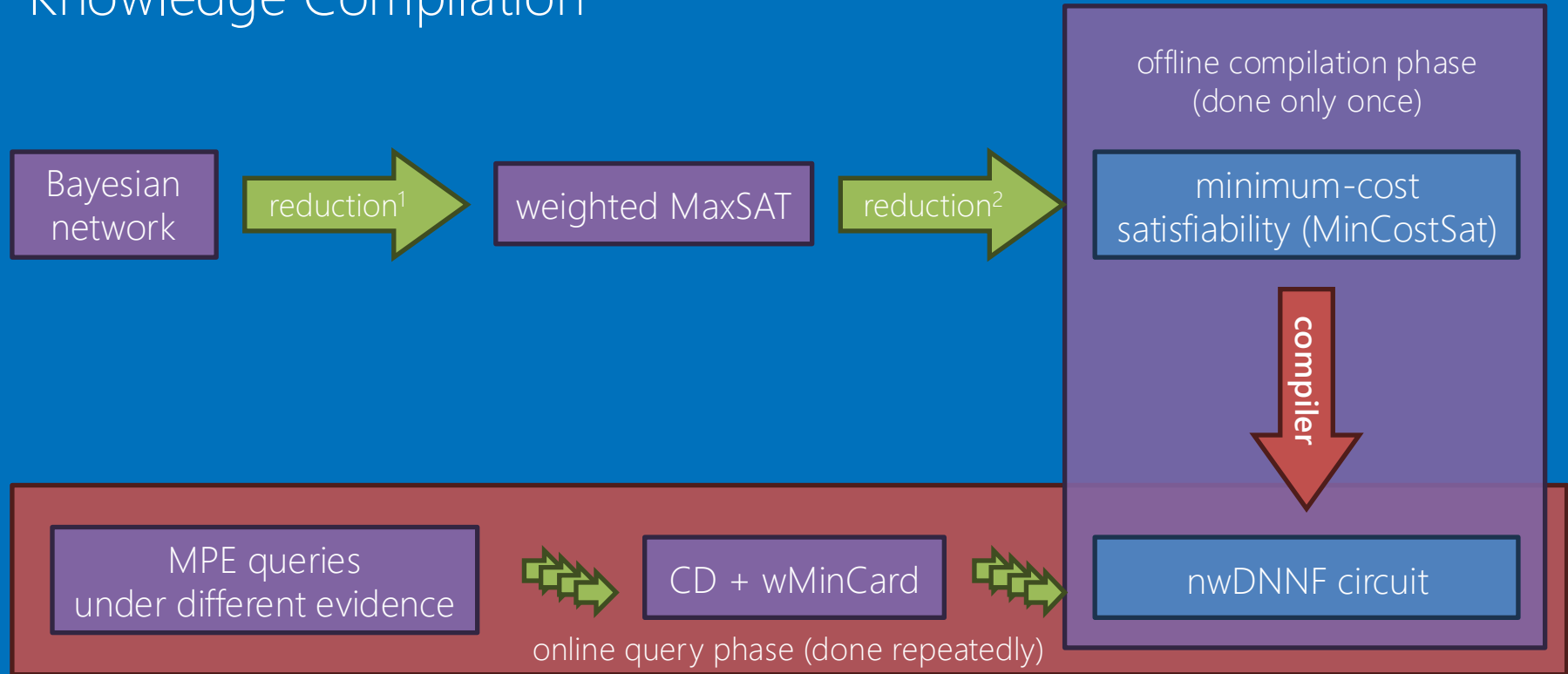
✓ polytime    ✗ not polytime unless  $P = NP$

\* **our contributions**

<sup>1</sup> DARWICHE, Adnan. Decomposable negation normal form. Journal of the ACM (JACM), 2001, 48.4: 608-647.

<sup>2</sup> DARWICHE, Adnan; MARQUIS, Pierre. Compiling propositional weighted bases. Artificial Intelligence, 2004, 157.1-2: 81-113.

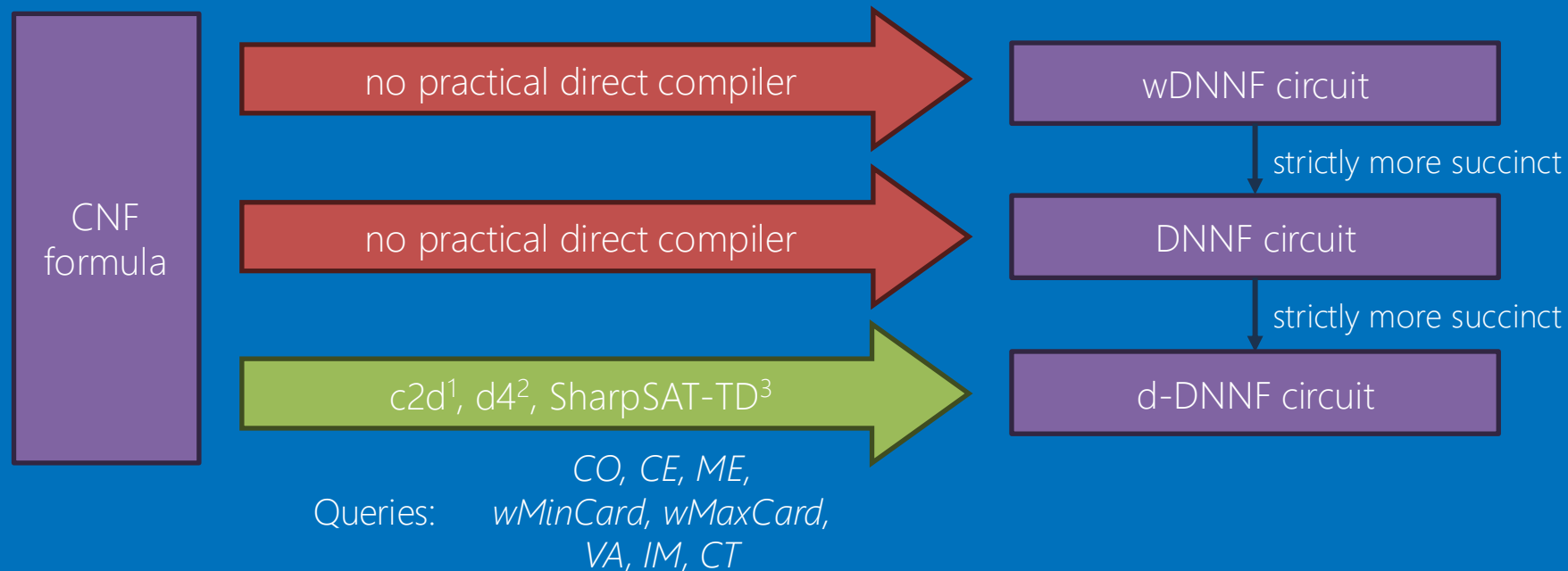
# Knowledge Compilation



<sup>1</sup> PARK, James D. Using weighted MAX-SAT engines to solve MPE. In: AAAI/IAAI. 2002. p. 682-687.

<sup>2</sup> PIPATSRISAWAT, Knot; DARWICHE, Adnan. Clone: Solving weighted Max-SAT in a reduced search space. In: AI 2007: Advances in Artificial Intelligence: 20th Australian Joint Conference on Artificial Intelligence, Gold Coast, Australia, December 2-6, 2007. Proceedings 20. Springer Berlin Heidelberg, 2007. p. 223-233.

# Knowledge Compilation (Before Our Research)

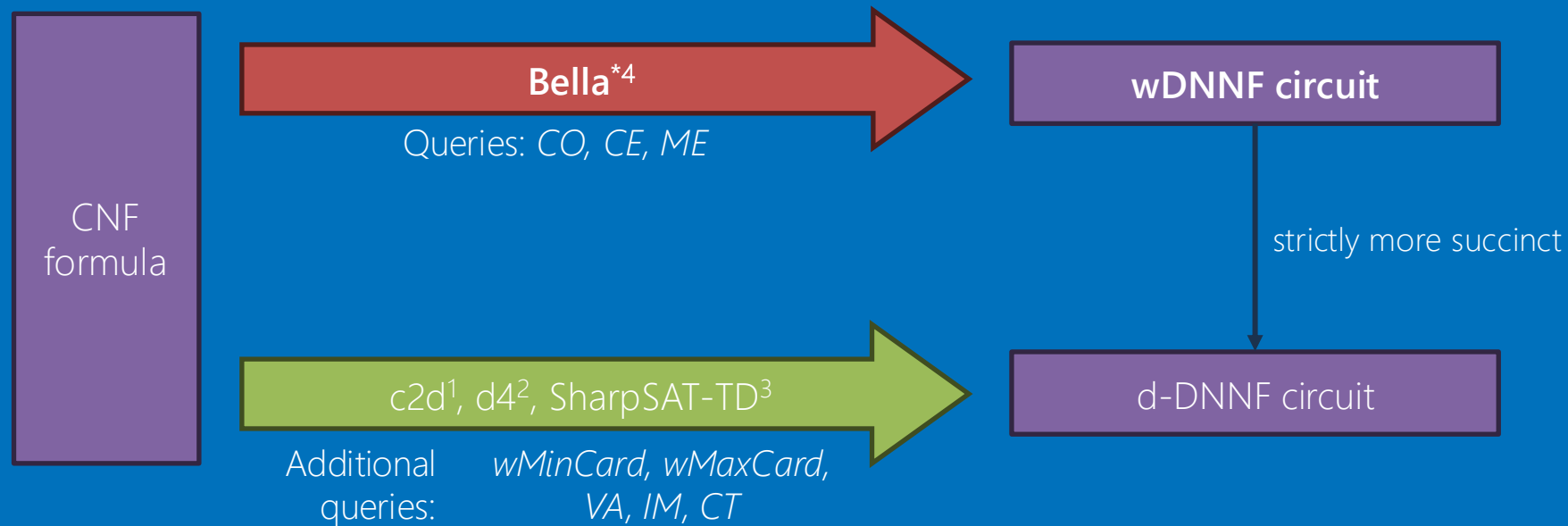


<sup>1</sup> DARWICHE, Adnan. New advances in compiling CNF to decomposable negation normal form. In: Proceedings of the 16th European Conference on Artificial Intelligence. 2004. p. 318-322.

<sup>2</sup> LAGNIEZ, Jean-Marie; MARQUIS, Pierre. An Improved Decision-DNNF Compiler. In: IJCAI. 2017. p. 667-673.

<sup>3</sup> KIESEL, Rafael; EITER, Thomas. Knowledge compilation and more with SharpSAT-TD. In: Proceedings of the International Conference on Principles of Knowledge Representation and Reasoning. 2023.

# Knowledge Compilers (**AAAI-24**<sup>4</sup>)



<sup>\*</sup> details in the extended abstract

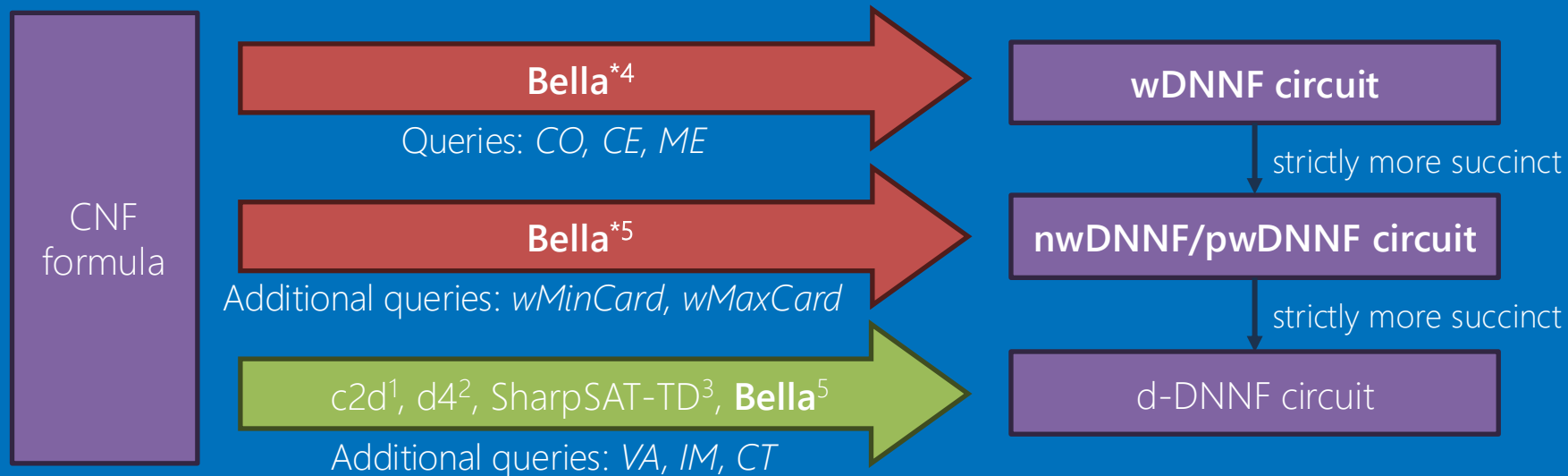
<sup>1</sup> DARWICHE, Adnan. New advances in compiling CNF to decomposable negation normal form. In: Proceedings of the 16th European Conference on Artificial Intelligence. 2004. p. 318-322.

<sup>2</sup> LAGNIEZ, Jean-Marie; MARQUIS, Pierre. An Improved Decision-DNNF Compiler. In: IJCAI. 2017. p. 667-673.

<sup>3</sup> KIESEL, Rafael; EITER, Thomas. Knowledge compilation and more with SharpSAT-TD. In: Proceedings of the International Conference on Principles of Knowledge Representation and Reasoning. 2023.

<sup>4</sup> ILLNER, Petr; KUČERA, Petr. A compiler for weak decomposable negation normal form. In: Proceedings of the AAAI Conference on Artificial Intelligence. 2024. p. 10562-10570.

# Knowledge Compilers (AAAI-25<sup>5</sup>)



<sup>\*</sup> details in the extended abstract

<sup>1</sup> DARWICHE, Adnan. New advances in compiling CNF to decomposable negation normal form. In: Proceedings of the 16th European Conference on Artificial Intelligence. 2004. p. 318-322.

<sup>2</sup> LAGNIEZ, Jean-Marie; MARQUIS, Pierre. An Improved Decision-DNNF Compiler. In: IJCAI. 2017. p. 667-673.

<sup>3</sup> KIESEL, Rafael; EITER, Thomas. Knowledge compilation and more with SharpSAT-TD. In: Proceedings of the International Conference on Principles of Knowledge Representation and Reasoning. 2023.

<sup>4</sup> ILLNER, Petr; KUČERA, Petr. A compiler for weak decomposable negation normal form. In: Proceedings of the AAAI Conference on Artificial Intelligence. 2024. p. 10562-10570.

<sup>5</sup> ILLNER, Petr. New compilation languages based on restricted weak decomposability. In: Proceedings of the AAAI Conference on Artificial Intelligence. 2025. p. 14987-14996.

# Enumeration of Valid Partial Configurations (*FO*, *ME*) – wDNNF Circuits

| Configuration instance <sup>1</sup> | wDNNF circuit |                  |          | decision-DNNF circuit |             |   | Improvements**      |                |
|-------------------------------------|---------------|------------------|----------|-----------------------|-------------|---|---------------------|----------------|
|                                     | Bella         |                  |          | D4                    |             |   | speed-up factor (×) | size reduction |
|                                     | time (s)      | size             | #        | time (s)              | size        | # |                     |                |
| C202_FS                             | <b>66.3</b>   | <b>4 245 485</b> | <b>5</b> | 1 256.2               | 154 862 147 | 4 | <b>18.9×</b>        | <b>97.26%</b>  |
| C202_FW                             | <b>70.7</b>   | <b>7 344 961</b> | <b>5</b> | —*                    | —*          | 0 | <b>N/A</b>          | <b>N/A</b>     |
| C210_FS                             | <b>33.2</b>   | <b>4 300 692</b> | <b>5</b> | 1 065.3               | 240 087 456 | 5 | <b>32.1×</b>        | <b>98.21%</b>  |
| C210_FW                             | <b>56.7</b>   | <b>6 429 027</b> | <b>5</b> | 4 048.2               | 715 624 258 | 2 | <b>71.4×</b>        | <b>99.10%</b>  |

Compilation times (in seconds), circuit sizes and the number of successful runs (**maximum is five**) for the four **most challenging small-scale automotive configuration instances from Mercedes-Benz**.

\* timeout after 2 hours

\*\* computed from successful runs only

<sup>1</sup> <https://www.cril.univ-artois.fr/kc/benchmarks.html>

# MPEs in Two-Layer BNs with Large Domains – nwDNNF Circuits

| #nodes      | density | domain size | nwDNNF circuits (Bella) |                   |           | decision-DNNF circuits (D4) |            |   |
|-------------|---------|-------------|-------------------------|-------------------|-----------|-----------------------------|------------|---|
|             |         |             | time (s)                | size              | #         | time (s)                    | size       | # |
| 5<br>:<br>5 | 100%    | 7           | <b>108</b>              | <b>1 909 132</b>  | <b>1</b>  | 590                         | 6 770 407  | 1 |
|             |         | 8           | <b>459</b>              | <b>4 209 175</b>  | <b>1</b>  | 3 296                       | 15 836 293 | 1 |
|             |         | 9           | <b>1 884</b>            | <b>8 471 321</b>  | <b>1</b>  | —*                          | —*         | 0 |
|             | 80%     | 13          | <b>664</b>              | <b>19 509 851</b> | <b>10</b> | 6 625                       | 34 465 982 | 1 |
|             |         | 14          | <b>1 492</b>            | <b>32 614 471</b> | <b>10</b> | —*                          | —*         | 0 |
|             |         | 15          | <b>3 255</b>            | <b>49 867 671</b> | <b>10</b> | —*                          | —*         | 0 |
|             | 60%     | 20          | <b>856</b>              | <b>29 159 870</b> | <b>10</b> | 3 255                       | 26 635 851 | 8 |
|             |         | 25          | <b>2 027</b>            | <b>34 843 482</b> | <b>10</b> | —*                          | —*         | 0 |
|             |         | 28          | <b>3 796</b>            | <b>52 154 815</b> | <b>10</b> | —*                          | —*         | 0 |

Compilation times (in seconds), circuit sizes, and the number of successfully compiled instances - **maximum is ten** (resp. **one**) for 60% and 80% densities (resp. 100% density).

\* timeout after 2 (resp. 6) hours

# Isomorphism-Aware Exact Model Counting

# Motivation

$$\psi_1: (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_3 \vee \neg x_4) \wedge (x_1 \vee \neg x_3 \vee x_4)$$

# Motivation

$$\psi_1: (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_3 \vee \neg x_4) \wedge (x_1 \vee \neg x_3 \vee x_4)$$

$$\#\psi_1 = 6$$

# Motivation

$$\psi_1 : (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_3 \vee \neg x_4) \wedge (x_1 \vee \neg x_3 \vee x_4)$$

$$\#\psi_1 = 6$$

$$\psi_2 : (x_5 \vee x_6) \wedge (\neg x_5 \vee x_7) \wedge (x_7 \vee \neg x_8) \wedge (x_5 \vee \neg x_7 \vee x_8)$$

# Motivation

$$\psi_1 : (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_3 \vee \neg x_4) \wedge (x_1 \vee \neg x_3 \vee x_4) \quad \#\psi_1 = 6$$

$$\psi_2 : (x_5 \vee x_6) \wedge (\neg x_5 \vee x_7) \wedge (x_7 \vee \neg x_8) \wedge (x_5 \vee \neg x_7 \vee x_8) \quad \#\psi_2 = 6$$

$$\{ x_1 \mapsto x_5, x_2 \mapsto x_6, x_3 \mapsto x_7, x_4 \mapsto x_8 \}$$

**variable renaming does NOT matter**

# Motivation

$$\psi_1 : (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_3 \vee \neg x_4) \wedge (x_1 \vee \neg x_3 \vee x_4) \quad \#\psi_1 = 6$$

$$\psi_2 : (x_5 \vee x_6) \wedge (\neg x_5 \vee x_7) \wedge (x_7 \vee \neg x_8) \wedge (x_5 \vee \neg x_7 \vee x_8) \quad \#\psi_2 = 6$$

$$\{ x_1 \mapsto x_5, x_2 \mapsto x_6, x_3 \mapsto x_7, x_4 \mapsto x_8 \}$$

**variable renaming does NOT matter**

$$\psi_3 : (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_1 \vee x_3 \vee \neg x_4)$$

# Motivation

$$\psi_1 : (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3) \wedge (x_3 \vee \neg x_4) \wedge (x_1 \vee \neg x_3 \vee x_4) \quad \#\psi_1 = 6$$

$$\psi_2 : (x_5 \vee x_6) \wedge (\neg x_5 \vee x_7) \wedge (x_7 \vee \neg x_8) \wedge (x_5 \vee \neg x_7 \vee x_8) \quad \#\psi_2 = 6$$

$$\{ x_1 \mapsto x_5, x_2 \mapsto x_6, x_3 \mapsto x_7, x_4 \mapsto x_8 \}$$

**variable renaming does NOT matter**

$$\psi_3 : (\neg x_1 \vee \neg x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_3 \vee x_4) \wedge (\neg x_1 \vee x_3 \vee \neg x_4) \quad \#\psi_3 = 6$$

**consistent polarity flips do NOT matter**

# Is detecting isomorphism worth the effort?

... it can be expensive

# Cara: An Isomorphism-Aware Model Counter – Key Idea

**Computing sample moments:** Describe each literal by its occurrence count and the sample moments of the sizes of the clauses containing it.

**Renaming variables/literals:** Order the variables and choose their polarities based on these statistics, then rename them accordingly.

**Reusing results:** When a renamed subformula matches a cached one, reuse its model count.

Details are provided in the extended abstract.

# Functional Pigeonhole Principle (FPHP) ...a prototypical highly symmetric problem

| Instance <sup>1</sup> | #pigeons | #holes |
|-----------------------|----------|--------|
| fphp-010-020          | 10       | < 20   |
| fphp-015-020          | 15       | < 20   |

## Runtime [s]

| Instance <sup>1</sup> | isomorphism-aware model counters |                              | standard model counters   |                                 |                        |
|-----------------------|----------------------------------|------------------------------|---------------------------|---------------------------------|------------------------|
|                       | <i>Cara</i> <sup>2</sup>         | <i>SymGanak</i> <sup>3</sup> | <i>Ganak</i> <sup>4</sup> | <i>SharpSAT-TD</i> <sup>5</sup> | <i>D4</i> <sup>6</sup> |
| fphp-010-020          | <b>3.04</b>                      | <b>2.37</b>                  | 3 176.17                  | — <sup>*</sup>                  | — <sup>*</sup>         |
| fphp-015-020          | <b>14.61</b>                     | <b>7.03</b>                  | — <sup>*</sup>            | — <sup>*</sup>                  | — <sup>*</sup>         |

<sup>1</sup> <https://www.cril.univ-artois.fr/kc/benchmarks.html>

<sup>2</sup> <https://github.com/llnec/CaraSolver>

<sup>3</sup> <https://github.com/meelgroup/ganak/tree/symganak>

<sup>4</sup> <https://github.com/meelgroup/ganak>

<sup>5</sup> <https://github.com/Laakeri/sharpsat-td>

<sup>6</sup> <https://github.com/crilab/d4v2>

\* timeout after 1 hour  
(matching the timeout used in the MCC 2025)

# *Cara*<sup>1\*</sup> VS *SymGanak*<sup>2\*\*,\*\*\*</sup>

\* uses **Cara caching scheme**

\*\* no preprocessing used (default settings)

\*\*\* uses **nauty**<sup>3</sup> to compute **graph canonical labellings**

<sup>1</sup> <https://github.com/llner/CaraSolver>

<sup>2</sup> <https://github.com/meelgroup/ganak/tree/symganak>

<sup>3</sup> <https://users.cecs.anu.edu.au/~bdm/nauty/>

# Experiments

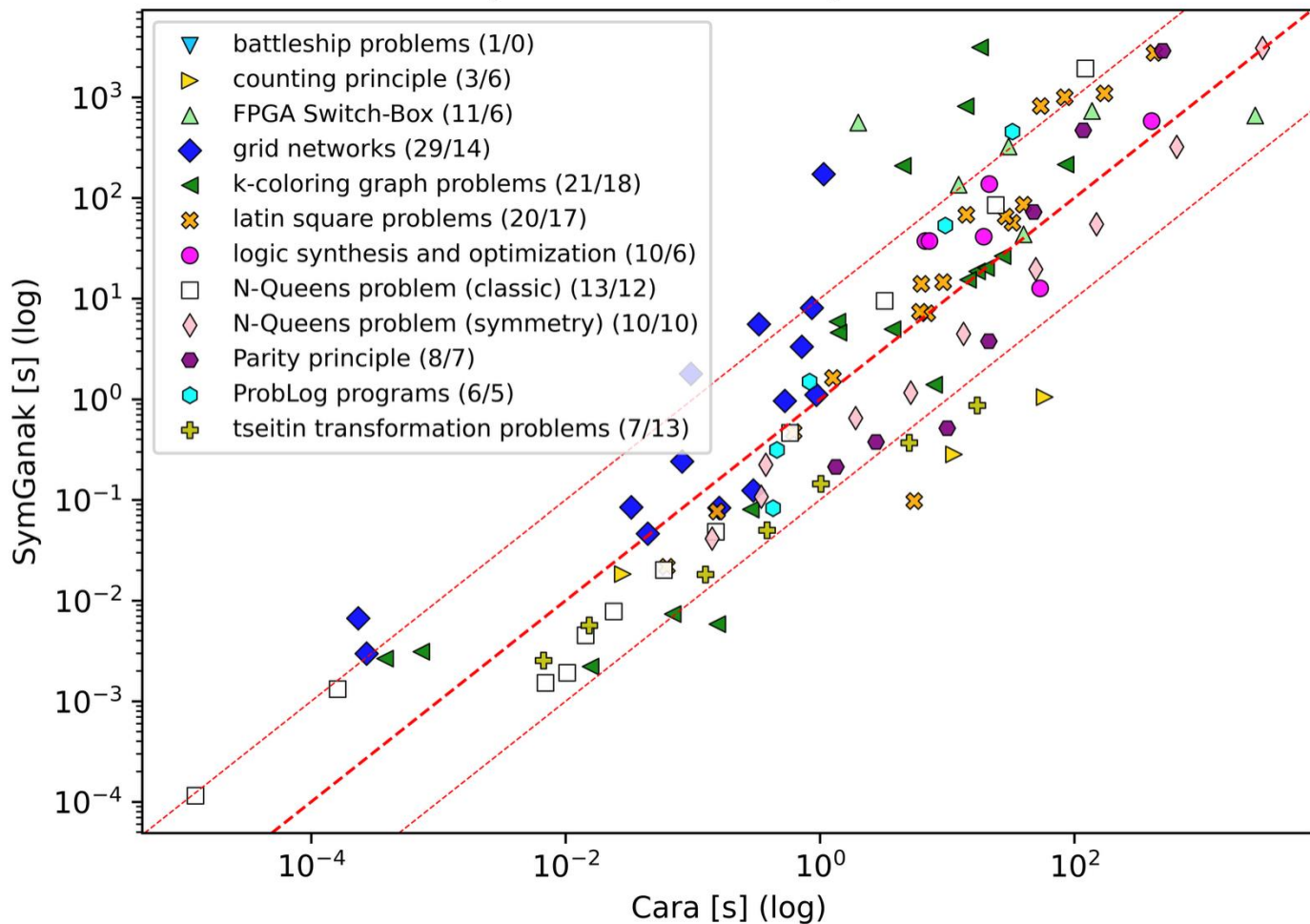
combinatorial benchmark

The time and memory limits were set to match those used in the Model Counting Competition 2025, namely **1 hour** and **32 GB**, respectively.

#solved  
instances

|          |     |
|----------|-----|
| Cara     | 139 |
| SymGanak | 114 |
| <hr/>    |     |
| Total    | 211 |

Solving time: Cara (139) vs SymGanak (114)



# Experiments

standard benchmark

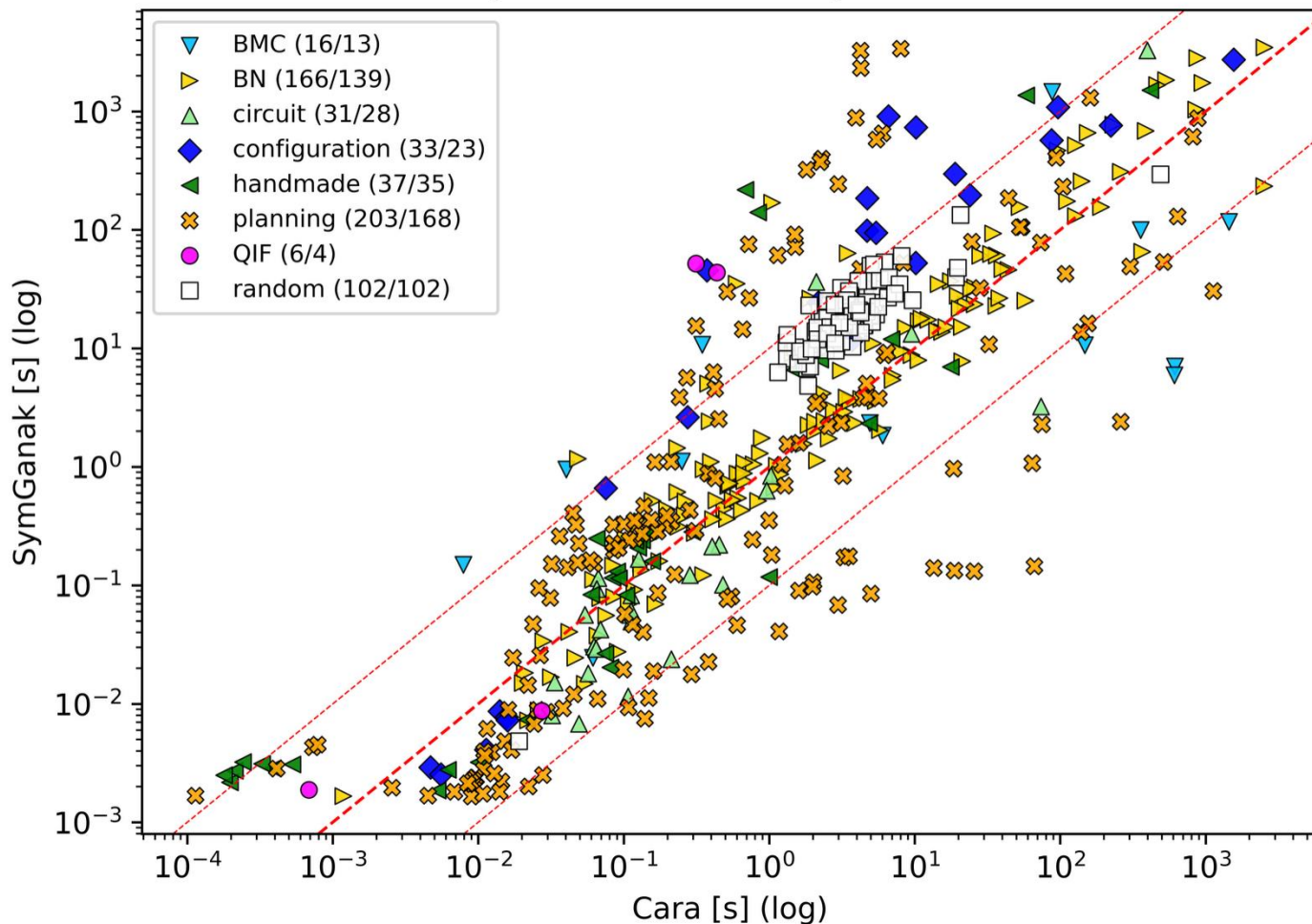
The time and memory limits were set to match those used in the Model Counting Competition 2025, namely **1 hour** and **32 GB**, respectively.

#solved  
instances

|          |     |
|----------|-----|
| Cara*    | 594 |
| SymGanak | 512 |
| <hr/>    |     |
| Total    | 701 |

\* solves every instance  
SymGanak solves

Solving time: Cara (594) vs SymGanak (512)



***Cara***<sup>1</sup> + ***Arjun***<sup>\*2</sup>  
VS  
***Ganak***<sup>3</sup>

<sup>1</sup> <https://github.com/IlIner/CaraSolver>

<sup>2</sup> <https://github.com/meelgroup/arjun>

<sup>3</sup> <https://github.com/meelgroup/ganak>

\* ensuring identical preprocessing

# Experiments

standard benchmark

The time and memory limits were set to match those used in the Model Counting Competition 2025, namely **1 hour** and **32 GB**, respectively.

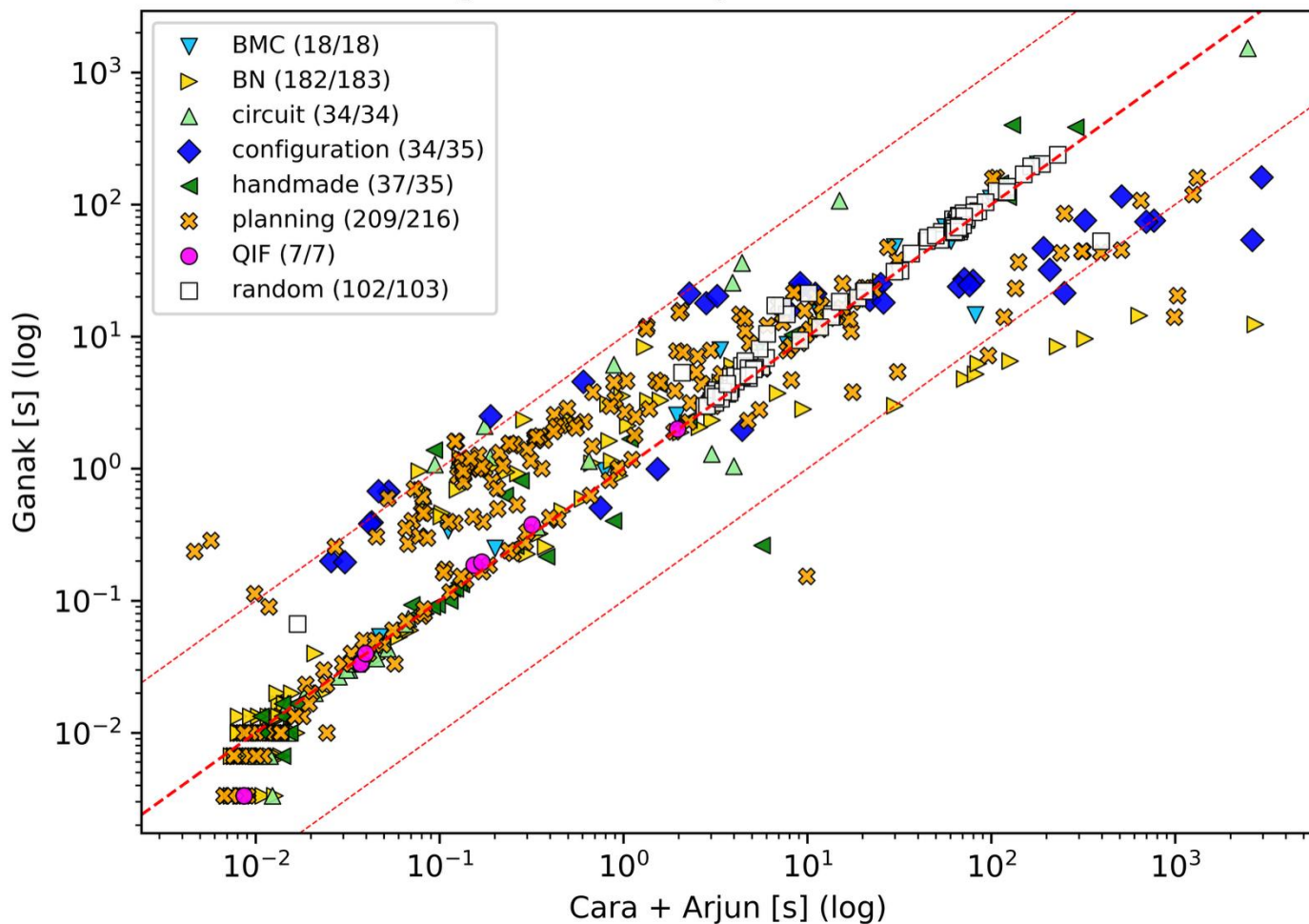
#solved  
instances

Cara 623

Ganak 631

Total 701

Solving time: Cara + Arjun (623) vs Ganak (631)



# Model Counting Competition<sup>1</sup> 2025 – Track 1 (Exact)

| Place | Model counter               | Author(s)                                      | (#)        | Solved    |
|-------|-----------------------------|------------------------------------------------|------------|-----------|
| 1     | Ganak                       | Mate Soos, Kuldeep S. Meel                     | (2)        | 60        |
| 2     | <b>Cara</b> <sup>*,**</sup> | <b>Petr Illner</b>                             | <b>(1)</b> | <b>50</b> |
|       | D4                          | Jean-Marie Lagniez, Pierre Marquis             | (2)        | 50        |
| 4     | SharpSAT-TD-CH              | Yipei Deng, Junping Zhou, Jiaxin Liang, Le Xin | (4)        | 47        |
|       | SharpSAT-TD                 | Tuukka Korhonen, Matti Järvisalo               | (2)        | 47        |
| 6     | MTMC                        | Ivor Spence                                    | (1)        | 39        |
| 7     | GPMC                        | Kenta Ito, Kenji Hashimoto                     | (2)        | 38        |

\* received the **Best Newcomer** notable mention

\*\* **first-ever isomorphism-aware model counter** to participate in the competition

<sup>1</sup> [https://mccompetition.org/2025/mc\\_description](https://mccompetition.org/2025/mc_description)

# Model Counting Competition<sup>1</sup> 2026 – Track 1 (Exact)

| Place    | Model counter | Author(s)                              | (#)        | Solved    |
|----------|---------------|----------------------------------------|------------|-----------|
| 1        | D4            | Pierre Marquis, Jean-Marie Lagniez     | (2)        | 63        |
| 2        | Ganak         | Mate Soos, Kuldeep S. Meel, Jiong Yang | (3)        | 62        |
| <b>3</b> | <b>Cara</b>   | <b>Petr Illner</b>                     | <b>(1)</b> | <b>59</b> |
| 4        | GPMC          | Kenta Ito, Kenji Hashimoto             | (2)        | 57        |
| 5        | TiDiDi        | Guy Van den Broeck, Florent Capelli    | (2)        | 56        |

MCC-25 included some highly symmetric instances; MCC-26 included none.  
Hence, Cara remains competitive beyond highly symmetric benchmarks.

<sup>1</sup> [https://mccompetition.org/2026/mc\\_description](https://mccompetition.org/2026/mc_description)

# Knowledge Representation and Knowledge Compilation

## Conference Papers

Illner, Petr; Kučera, Petr. *A Compiler for Weak Decomposable Negation Normal Form*.  
Proceedings of the **AAAI** Conference on Artificial Intelligence 2024.

Illner, Petr. *New Compilation Languages Based on Restricted Weak Decomposability*.  
Proceedings of the **AAAI** Conference on Artificial Intelligence 2025.

**Selected for oral presentation (4.6% of submissions).**

## Software

Illner, Petr. **Bella**: *A knowledge compiler for wDNF, pwDNF, nwDNF, and (s)d-DNF circuits*.

# Isomorphism-Aware Exact Model Counting

## Competitions

Illner, Petr. Cara, **Model Counting Competition 2025**.  
**Second place** in Track 1: Model Counting, Ranking A (exact); **Notable Mention: Best Newcomer**.

Illner, Petr. Cara, **Model Counting Competition 2026**.  
**Third place** in Track 1: Model Counting, Ranking A (exact).

## Workshop Contributions

Illner, Petr. Cara: *An Isomorphism-Based #SAT Solver*.  
**Workshop on Counting, Sampling, and Synthesis 2025**.

## Software

Illner, Petr. **Cara**: *An isomorphism-aware #SAT solver*.

## Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

# Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID |
|-------|-------|--------------|---------|----------------|-------|--------------|----|
| $x_3$ | 2     | 2            | 1.5     | 1.5            | 2     | 2            | 0  |
| $x_4$ | 2     | 2            | 1.5     | 1.5            | 2     | 2            | 1  |
| $x_5$ | 2     | 2            | 1.5     | 1.5            | 2     | 2            | 0  |
| $x_6$ | 2     | 2            | 1.5     | 1.5            | 2     | 2            | 1  |

$C_l$  = the number of clauses containing literal  $l$

$\mu_l$  = the average size of clauses containing literal  $l$

$m_l$  = the average squared size of clauses containing literal  $l$

$C_x < C_{\neg x}$  = consistent polarity flip

# Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID |
|-------|-------|--------------|---------|----------------|-------|--------------|----|
| $x_3$ |       |              |         |                |       |              | 3  |
| $x_4$ |       |              |         |                |       |              |    |
| $x_5$ |       |              |         |                |       |              |    |
| $x_6$ |       |              |         |                |       |              |    |

$C_l$  = the number of clauses containing literal  $l$

$\mu_l$  = the average size of clauses containing literal  $l$

$m_l$  = the average squared size of clauses containing literal  $l$

$C_x < C_{\neg x}$  = consistent polarity flip

# Cara Caching Scheme

$$\psi = (\underline{x_3} \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (\underline{x_3} \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID |
|-------|-------|--------------|---------|----------------|-------|--------------|----|
| $x_3$ |       |              |         |                |       |              | 3  |
| $x_4$ |       |              |         |                |       |              |    |
| $x_5$ |       |              |         |                |       |              |    |
| $x_6$ |       |              |         |                |       |              |    |

$C_l$  = the number of clauses containing literal  $l$

$\mu_l$  = the average size of clauses containing literal  $l$

$m_l$  = the average squared size of clauses containing literal  $l$

$C_x < C_{\neg x}$  = consistent polarity flip

# Cara Caching Scheme

$$\psi = (\underline{x_3} \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (\underline{x_3} \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID |
|-------|-------|--------------|---------|----------------|-------|--------------|----|
| $x_3$ | 2     |              | 3       |                | 10    |              | 3  |
| $x_4$ |       |              |         |                |       |              |    |
| $x_5$ |       |              |         |                |       |              |    |
| $x_6$ |       |              |         |                |       |              |    |

$C_l$  = the number of clauses containing literal  $l$

$\mu_l$  = the average size of clauses containing literal  $l$

$m_l$  = the average squared size of clauses containing literal  $l$

$C_x < C_{\neg x}$  = consistent polarity flip

# Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg \underline{x_3} \vee x_4 \vee \neg x_5) \wedge (\neg \underline{x_3} \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID |
|-------|-------|--------------|---------|----------------|-------|--------------|----|
| $x_3$ | 2     |              | 3       |                | 10    |              | 3  |
| $x_4$ |       | 2            |         | 3              |       | 10           | 3  |
| $x_5$ | 2     |              | 3       |                | 10    |              | 3  |
| $x_6$ |       | 2            |         | 3              |       | 10           | 3  |

$C_l$  = the number of clauses containing literal  $l$

$\mu_l$  = the average size of clauses containing literal  $l$

$m_l$  = the average squared size of clauses containing literal  $l$

$C_x < C_{\neg x}$  = consistent polarity flip

# Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg \underline{x_3} \vee x_4 \vee \neg x_5) \wedge (\neg \underline{x_3} \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID |
|-------|-------|--------------|---------|----------------|-------|--------------|----|
| $x_3$ | 2     | <b>2</b>     | 3       | <b>3</b>       | 10    | <b>9</b>     | 3  |
| $x_4$ |       |              |         |                |       |              |    |
| $x_5$ |       |              |         |                |       |              |    |
| $x_6$ |       |              |         |                |       |              |    |

$C_l$  = the number of clauses containing literal  $l$

$\mu_l$  = the average size of clauses containing literal  $l$

$m_l$  = the average squared size of clauses containing literal  $l$

$C_x < C_{\neg x}$  = consistent polarity flip

# Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID |
|-------|-------|--------------|---------|----------------|-------|--------------|----|
| $x_3$ | 2     | 2            | 3       | 3              | 10    | 9            | 3  |
| $x_4$ | 2     | 2            | 3       | 3              | 9     | 10           | 4  |
| $x_5$ | 1     | 2            | 2       | 3.5            | 4     | 12.5         | 5  |
| $x_6$ | 2     | 1            | 3.5     | 2              | 12.5  | 4            | 6  |

$C_l$  = the number of clauses containing literal  $l$

$\mu_l$  = the average size of clauses containing literal  $l$

$m_l$  = the average squared size of clauses containing literal  $l$

$C_x < C_{\neg x}$  = consistent polarity flip

# Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID | Order    | $C_l$ = the number of clauses containing literal $l$<br>$\mu_l$ = the average size of clauses containing literal $l$<br>$m_l$ = the average squared size of clauses containing literal $l$<br>$C_x < C_{\neg x}$ = consistent polarity flip |
|-------|-------|--------------|---------|----------------|-------|--------------|----|----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $x_3$ | 2     | 2            | 3       | 3              | 10    | 9            | 3  | <b>4</b> |                                                                                                                                                                                                                                             |
| $x_4$ | 2     | 2            | 3       | 3              | 9     | 10           | 4  | <b>3</b> |                                                                                                                                                                                                                                             |
| $x_5$ | 1     | 2            | 2       | 3.5            | 4     | 12.5         | 5  | <b>1</b> |                                                                                                                                                                                                                                             |
| $x_6$ | 2     | 1            | 3.5     | 2              | 12.5  | 4            | 6  | <b>2</b> |                                                                                                                                                                                                                                             |

# Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID | Order    |
|-------|-------|--------------|---------|----------------|-------|--------------|----|----------|
| $x_3$ | 2     | 2            | 3       | 3              | 10    | 9            | 3  | <b>4</b> |
| $x_4$ | 2     | 2            | 3       | 3              | 9     | 10           | 4  | <b>3</b> |
| $x_5$ | 1     | 2            | 2       | 3.5            | 4     | 12.5         | 5  | <b>1</b> |
| $x_6$ | 2     | 1            | 3.5     | 2              | 12.5  | 4            | 6  | <b>2</b> |

$$M = \{ 3 \mapsto 4, \\ 4 \mapsto 3, \\ 5 \mapsto 1, \\ 6 \mapsto 2 \}$$

# Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID | Order    |
|-------|-------|--------------|---------|----------------|-------|--------------|----|----------|
| $x_3$ | 2     | 2            | 3       | 3              | 10    | 9            | 3  | <b>4</b> |
| $x_4$ | 2     | 2            | 3       | 3              | 9     | 10           | 4  | <b>3</b> |
| $x_5$ | 1     | < 2          | 2       | 3.5            | 4     | 12.5         | 5  | <b>1</b> |
| $x_6$ | 2     | 1            | 3.5     | 2              | 12.5  | 4            | 6  | <b>2</b> |

$$M = \{ 3 \mapsto 4, \\ 4 \mapsto 3, \\ 5 \mapsto -1, \\ 6 \mapsto 2 \}$$

# Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID | Order    |
|-------|-------|--------------|---------|----------------|-------|--------------|----|----------|
| $x_3$ | 2     | 2            | 3       | 3              | 10    | 9            | 3  | <b>4</b> |
| $x_4$ | 2     | 2            | 3       | 3              | 9     | 10           | 4  | <b>3</b> |
| $x_5$ | 1 <   | 2            | 2       | 3.5            | 4     | 12.5         | 5  | <b>1</b> |
| $x_6$ | 2     | 1            | 3.5     | 2              | 12.5  | 4            | 6  | <b>2</b> |

$$M = \{ 3 \mapsto 4, \\ 4 \mapsto 3, \\ 5 \mapsto -1, \\ 6 \mapsto 2 \}$$

$$M(\psi) = (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2) \wedge (\neg x_4 \vee x_3 \vee x_1) \wedge (\neg x_4 \vee x_3 \vee x_2) \wedge (x_4 \vee \neg x_3 \vee x_1 \vee x_2)$$

# Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID | Order    |
|-------|-------|--------------|---------|----------------|-------|--------------|----|----------|
| $x_3$ | 2     | 2            | 3       | 3              | 10    | 9            | 3  | <b>4</b> |
| $x_4$ | 2     | 2            | 3       | 3              | 9     | 10           | 4  | <b>3</b> |
| $x_5$ | 1     | < 2          | 2       | 3.5            | 4     | 12.5         | 5  | <b>1</b> |
| $x_6$ | 2     | 1            | 3.5     | 2              | 12.5  | 4            | 6  | <b>2</b> |

$$M = \{ 3 \mapsto 4, \\ 4 \mapsto 3, \\ 5 \mapsto -1, \\ 6 \mapsto 2 \}$$

$$M(\psi) = (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2) \wedge (\neg x_4 \vee x_3 \vee x_1) \wedge (\neg x_4 \vee x_3 \vee x_2) \wedge (x_4 \vee \neg x_3 \vee x_1 \vee x_2)$$

$$r(\psi) = \underline{-1, 4, 0, -2, -3, 0, 1, 3, -4, 0, 2, 3, -4, 0, 1, 2, -3, 4, 0}$$

sorted residual clauses

# Cara Caching Scheme

$$\psi = (x_3 \vee x_5) \wedge (\neg x_4 \vee \neg x_6) \wedge (\neg x_3 \vee x_4 \vee \neg x_5) \wedge (\neg x_3 \vee x_4 \vee x_6) \wedge (x_3 \vee \neg x_4 \vee \neg x_5 \vee x_6)$$

|       | $C_x$ | $C_{\neg x}$ | $\mu_x$ | $\mu_{\neg x}$ | $m_x$ | $m_{\neg x}$ | ID | Order    |                                                                                |
|-------|-------|--------------|---------|----------------|-------|--------------|----|----------|--------------------------------------------------------------------------------|
| $x_3$ | 2     | 2            | 3       | 3              | 10    | 9            | 3  | <b>4</b> | $M = \{ 3 \mapsto 4,$<br>$4 \mapsto 3,$<br>$5 \mapsto -1,$<br>$6 \mapsto 2 \}$ |
| $x_4$ | 2     | 2            | 3       | 3              | 9     | 10           | 4  | <b>3</b> |                                                                                |
| $x_5$ | 1     | < 2          | 2       | 3.5            | 4     | 12.5         | 5  | <b>1</b> |                                                                                |
| $x_6$ | 2     | 1            | 3.5     | 2              | 12.5  | 4            | 6  | <b>2</b> |                                                                                |

$$M(\psi) = (x_4 \vee \neg x_1) \wedge (\neg x_3 \vee \neg x_2) \wedge (\neg x_4 \vee x_3 \vee x_1) \wedge (\neg x_4 \vee x_3 \vee x_2) \wedge (x_4 \vee \neg x_3 \vee x_1 \vee x_2)$$

$$r(\psi) = \underline{-1, 4, 0, -2, -3, 0, 1, 3, -4, 0, 2, 3, -4, 0, 1, 2, -3, 4, 0}$$

sorted residual clauses

**cache entry:** key =  $r(\psi)$ ,  
value =  $\#\psi = 5$